What Do Spikes Buy in Flow-Based Intrusion Detection? A Leakage-Controlled Study of an Ensemble Spiking Network on CSE-CIC-IDS2018

Özgür Uçar
ucar45ozgur@gmail.com

Abstract*—Spiking neural networks (SNNs) are often proposed for network intrusion detection because event-driven hardware could run them cheaply at the network edge. Most reported results, however, come from random row splits of benchmark datasets that contain many exact duplicates, and from comparisons that leave out strong tree ensembles. We revisit the question on CSE-CIC-IDS2018 (16.2 million flows, 13 classes) with a protocol built to avoid those problems. Rows that share an identical cleaned feature vector are kept in the same partition. Model fitting, probability calibration, alarm-threshold selection and testing use four disjoint partitions, and every model is judged at the same benign false-alarm budget. We compare an ensemble of three SNN branches (rate, latency and population coding, 86k parameters) against a parameter-matched multilayer perceptron, a control network that receives the same spike-encoded input without spiking dynamics, and three tree ensembles. At a 5% false-alarm budget the SNN reaches a macro-F1 of 0.769 ± 0.022 over three seeds. This is above the matched perceptron (0.756) but below the spike-input control (0.787), random forest (0.788), CatBoost (0.791) and XGBoost (0.808). Attack detection rates differ by less than one point across all models. A feature-group audit shows that most of the remaining error is not a modelling problem: 99.99% of FTP-BruteForce flows and nearly all DoS-SlowHTTPTest flows have feature vectors that also occur under another label. Under a standard 45 nm energy model the SNN needs about 17% less synaptic energy per flow than the matched perceptron, but the spike-input control needs six to eight times less than either. Our reading is that the input encoders, not the spiking dynamics, carry most of the benefit in this setting. We release code, frozen checkpoints and a containerised detector.

***Index Terms*—intrusion detection, spiking neural networks, CSE-CIC-IDS2018, data leakage, calibration, neuromorphic computing**

I. Introduction

Network intrusion detection systems (NIDS) that classify flow records are attractive because flow export is cheap and already deployed in most networks. Learning-based detectors trained on public benchmarks such as CSE-CIC-IDS2018 [1] routinely report accuracies above 99%. Spiking neural networks [2] have joined this line of work, motivated by the low power that neuromorphic chips such as Loihi [3] promise for always-on monitoring.

Two problems make many of these numbers hard to interpret. The first is the data. The CIC datasets contain labelling and feature-extraction errors [4], [5], and a large share of their rows are exact duplicates. When a random row split puts copies of the same vector into both training and test sets, the test score partly measures memorisation [6], [7]. The second is the comparison. SNN papers often compare against a convolutional or recurrent network trained in a similar way, while gradient-boosted trees, which remain very hard to beat on tabular data [8], are left out. Reporting accuracy at the default argmax decision also hides the operating point that matters to an analyst: how many attacks are caught at a fixed rate of false alarms.

This paper asks a narrow question. If the evaluation controls duplicate leakage, calibrates every model the same way and fixes the false-alarm budget, does an ensemble SNN offer an accuracy or efficiency advantage on CSE-CIC-IDS2018? Our contributions are:

- A leakage-controlled protocol for CSE-CIC-IDS2018 in which identical cleaned feature vectors never cross partitions, with separate partitions for model selection, probability calibration, alarm calibration and test.
- We compare a three-branch ensemble SNN against a parameter-matched perceptron, a *spike-input control* that isolates the encoders from the spiking dynamics, and three tree ensembles, over three seeds and three false-alarm budgets.
- We audit duplicated feature vectors to bound the error any model can reach on this representation, which explains the two classes on which every model fails.
- Operation counts for the neural models and a containerised detector that serves the frozen checkpoints for testing on new flow records.

The headline result is mixed and, we think, useful. The SNN is a competent detector, but in this setting it neither matches the tree ensembles nor beats a non-spiking network that receives the same encoded input.

II. Related Work

Dataset quality. Engelen et al. [4] documented flow-construction and labelling errors in CICIDS2017, and Liu et al. [5] extended the analysis to CSE-CIC-IDS2018. Arp et al. [7] list sampling bias and data snooping among the most common pitfalls of learning-based security research,

and Sommer and Paxson [9] warned early that closed-world benchmark success rarely transfers to operational networks. We follow these recommendations in the split design and in how we report the remaining errors.

Spiking networks. Surrogate-gradient training [10] made it practical to train SNNs with backpropagation through time, and libraries such as snnTorch [11] brought this into standard deep-learning tooling. Tabular inputs must first be turned into spike trains. Rate, latency and population codes are the usual choices, and the choice of code can matter as much as the neuron model. Energy claims for SNNs normally rest on the fact that an accumulate costs less than a multiply-accumulate [12], which only pays off when spikes are sparse and time windows are short.

Calibration and operating points. Class-prior correction [13] and temperature scaling [14] are standard ways to fix probability estimates after training on a resampled set. Neyman–Pearson classification [15] frames detection as maximising power under a constraint on the false-positive rate. That is the framing we use for alarm thresholds.

III. Data and Protocol

A. Dataset and cleaning

We use the ten daily CSV files of CSE-CIC-IDS2018 as distributed on Kaggle. After removing repeated header lines and rows with unparseable timestamps, 16,232,929 flows remain. Original labels are mapped to 13 classes; the three web-attack labels (brute force, XSS and SQL injection) are merged into one *Web Attacks* class because together they have only 928 flows. Flow identifiers, IP addresses, source port and timestamp are dropped before learning. Non-finite values are replaced by zero, and we record how often this happens for each column.

B. Feature-group split

We compute a 128-bit fingerprint of each row's cleaned feature vector and treat all rows with the same fingerprint as one group. Groups, not rows, are assigned to the training, tuning, calibration and test partitions in a 60/10/10/20 ratio. A group is not a network session; it only guarantees that no test vector was seen verbatim during training. Classes with many duplicates shrink a great deal under this rule. DDoS-HOIC, for example, has 686,012 flows but only 198,861 distinct vectors. Table I gives the resulting support.

The training partition is capped at 500,000 rows. Benign rows are subsampled, and the rarest classes are kept in full. No synthetic oversampling is used. The calibration partition is split again into a part used to fit probability calibration and a part used only to choose alarm thresholds.

C. Preprocessing

Every preprocessing step is fitted on the training partition only. Constant columns are removed, columns with absolute skew above 5 get a signed $\log(1 + |x|)$ transform, and a quantile transformer maps each feature to $[0, 1]$. Seventy features remain.

TABLE I
Support per class after the feature-group split. Only training is capped.

Class	Train used	Tune	Cal.	Test
Benign	335,515	1,362,310	1,330,511	2,699,900
Bot	16,651	22,981	33,237	54,821
DDoS-HOIC	26,260	68,256	69,218	134,836
DDoS-LOIC-UDP	1,066	159	181	324
DDoS-LOIC-HTTP	23,516	57,929	57,319	115,366
DoS-GoldenEye	10,601	4,135	4,078	8,362
DoS-Hulk	20,762	46,577	45,539	92,601
DoS-SlowHTTPTest	14,486	6,487	568	11,441
DoS-Slowloris	6,422	1,053	1,064	2,451
FTP-BruteForce	16,781	1,879	6,095	7,049
Infiltration	13,497	16,205	16,432	32,470
SSH-Bruteforce	13,887	9,435	9,888	61,742
Web Attacks	556	87	106	179

D. Decision rule

All models output 13 class scores, which we treat as logits z (tree ensembles use log-probabilities). Because training was class-capped, we first add a prior offset $\log \pi_c^{\text{full}} - \log \pi_c^{\text{train}}$ [13]. A single temperature T_{cal} is then fitted by minimising negative log-likelihood on the probability-calibration rows [14]. The alarm score is the attack log-odds

$$s(x) = \log \sum_{c \neq \text{Benign}} e^{z_c / T_{\text{cal}}} - z_{\text{Benign}} / T_{\text{cal}}. \tag{1}$$

For a false-alarm budget τ, the threshold t_τ is the empirical $(1 - \tau)$ quantile of s over benign rows of the alarm-calibration partition. A flow raises an alarm if $s(x) > t_\tau$ and is then assigned the most probable attack class. We use $\tau \in \{1\%, 5\%, 10\%\}$ and fix $\tau = 5\%$ as the primary budget before testing. Test labels are never used to pick a threshold, a temperature or a checkpoint.

IV. Models

Ensemble SNN (E-SNN). Three branches share the same architecture and differ only in how the 70 features become spikes over $T = 12$ time steps. The *rate* branch uses a deterministic phase-accumulator code. The *temporal* branch emits a single spike whose latency decreases with feature value. The *population* branch expands each feature into four Gaussian receptive fields and rate-codes their activations. Each branch has two hidden layers of 128 and 64 leaky integrate-and-fire neurons with learnable decay, layer normalisation and dropout of 0.2, followed by a linear readout averaged over time. The three branch outputs are combined as 0.6 times a softmax-weighted average plus 0.4 times a small meta-network. The model has 86,013 parameters and is trained with surrogate gradients (fast sigmoid, slope 25) using snnTorch [11], for at most 35 epochs with early stopping on tuning macro-F1.

Matched ANN. A two-hidden-layer perceptron whose width is chosen by parameter count alone to match the E-SNN (85,945 parameters). It sees the quantile-scaled features directly.

TABLE II
TEST RESULTS AT THE 5% BENIGN FALSE-ALARM BUDGET (MEAN ± S.D., THREE SEEDS). ADR: ATTACK DETECTION RATE.

Model	Macro-F1	ADR	FPR
E-SNN	0.769 ± 0.022	0.959 ± 0.001	0.049
Matched ANN	0.756 ± 0.004	0.958 ± 0.000	0.050
Enc-ANN (control)	0.787 ± 0.017	0.963 ± 0.001	0.049
Random forest	0.788 ± 0.001	0.965 ± 0.000	0.050
CatBoost	0.791 ± 0.005	0.965 ± 0.000	0.049
XGBoost	$\mathbf{0.808 \pm 0.005}$	$\mathbf{0.967 \pm 0.000}$	0.049

Spike-input control (Enc-ANN). The same three fixed encoders and fusion head as the E-SNN, but each branch flattens the whole spike train and passes it through an ordinary feed-forward network of matched size (84,503 parameters). Comparing E-SNN with Enc-ANN separates the effect of the encoding from the effect of membrane dynamics.

Tree ensembles. Random forest [16], XGBoost [17] and CatBoost [18] with fixed, commonly used hyperparameters. We did not tune them, which if anything favours the neural models.

Every neural and tree model is trained with plain cross-entropy for three seeds (42, 43, 44). As an additional check, the E-SNN and matched ANN are also trained with class-weighted cross-entropy.

V. RESULTS

A. Detection at a fixed false-alarm budget

Table II reports test results at the primary 5% budget. The achieved test false-alarm rate is between 4.9% and 5.0% for every model, so the budget transfers well from calibration to test. The Kolmogorov–Smirnov distance between benign calibration and benign test scores is 0.006 for the E-SNN.

All six models catch about 96% of attack flows. Most of the separation shows up in macro-F1, which weights the rare and difficult classes equally. The E-SNN is 1.3 points above the matched ANN, which suggests the spike encoders help. It is 1.8 points below the spike-input control, though, and 3.9 points below XGBoost. Its seed variance is also the largest of all models: one of the three seeds reached 0.744. The ranking holds at the other budgets. At $\tau = 1\%$ the macro-F1 values are 0.789 (E-SNN), 0.771 (ANN), 0.816 (Enc-ANN), 0.824 (RF), 0.838 (CatBoost) and 0.848 (XGBoost). Class-weighted training did not help either neural model; the E-SNN dropped to 0.753 ± 0.004 and the ANN to 0.747 ± 0.003.

B. Where the errors are

Table III lists per-class F1. Nine of the thirteen classes are detected almost perfectly by every model. The differences between models come almost entirely from four classes: DoS-SlowHTTPTest, FTP-BruteForce, Infiltration and Web Attacks.

The feature-group audit explains three of these. Of 193,360 FTP-BruteForce flows, 193,353 fall into just 54 distinct vectors, and every one of those vectors also appears under another label. The reverse holds for DoS-SlowHTTPTest, where 139,884 of 139,890 flows share vectors with other classes. Both attacks leave short, nearly empty TCP connections that the flow extractor can summarise identically, and in that case no classifier working on these features can tell them apart. Infiltration is different. Its flows are spread over 140,610 groups, but 88,810 of them share a vector with benign traffic, which is consistent with the known labelling problems in this class [5]. Across the whole dataset, 68,467 groups carry conflicting labels, and a classifier that predicts the majority label of each group would still be wrong on 1.10% of rows. Any reported error below that floor on this representation is a sign of leakage.

TABLE III
PER-CLASS TEST F1 AT THE 5% BUDGET, MEAN OF THREE SEEDS. CLASSES WITH F1 ≥ 0.96 FOR ALL MODELS ARE OMITTED.

Class	E-SNN	ANN	Enc-ANN	RF	XGB
DoS-SlowHTTPTest	0.552	0.519	0.625	0.747	0.734
FTP-BruteForce	0.243	0.271	0.274	0.234	0.337
Infiltration	0.127	0.124	0.152	0.169	0.171
Web Attacks	0.360	0.171	0.393	0.319	0.464

TABLE IV
SYNAPTIC OPERATIONS PER FLOW AND A FIRST-ORDER ENERGY ESTIMATE. CPU TIME IS SINGLE-THREAD, BATCH 4,400.

Model	ACs	MACs	Est. energy	CPU time
E-SNN	339,801	3,328	0.32 μJ	57.1 μs
Matched ANN	0	84,420	0.39 μJ	1.4 μs
Enc-ANN	28,956	5,094	0.05 μJ	16.3 μs

C. Operation counts and runtime

We counted operations with forward hooks on 4,400 class-balanced flows. In a linear layer that receives binary input, each nonzero input contributes one accumulate (AC) per output neuron; any other linear layer contributes multiply-accumulates (MACs). Table IV reports counts per flow. The energy column applies the 45 nm estimates of Horowitz [12] (0.9 pJ per 32-bit AC, 4.6 pJ per MAC) and should be read only as a comparison between the models. It ignores membrane updates, memory traffic and encoding, which often dominate on real hardware.

The E-SNN's hidden neurons are silent 72% of the time, yet with 12 time steps and three branches it still performs four times as many synaptic events as the matched ANN performs MACs. The cheaper accumulate makes up for this, and the estimate ends about 17% below the ANN. The spike-input control is far cheaper than both, because it sees each binary input once instead of propagating it through recurrent membrane state at every time step. On a general-purpose CPU, where sparsity brings no saving, the E-SNN is about 40 times slower than the ANN. Even so, the service scores more than 15,000 flows per second on a single laptop CPU, which is enough for a small network segment.

VI. A Containerised Detector

To let others test the frozen models on their own captures, we package the seed-42 E-SNN and XGBoost checkpoints, the fitted preprocessor and the calibration records in a Docker image. A FastAPI service accepts CICFlowMeter CSV files or JSON batches of flows and returns a class, an alarm flag and the attack log-odds for each flow, using the same 5% threshold as Table II. Before loading, the service checks each checkpoint's SHA-256 digest against its calibration record. A replay script streams flows to the service in small batches to imitate a live sensor. On a class-balanced sample of 4,400 flows from four capture days the E-SNN raised alarms on every attack flow and on 1.3% of benign flows. That sample is drawn from the public data and may overlap the test partition, so it shows that the service works but does not count as a further evaluation.

VII. Discussion and Limitations

What the spikes buy. Within this protocol the useful part of the SNN appears to be its input encoding. The population and latency codes give a plain feed-forward network a better view of skewed flow features, and that network then beats the spiking version on both accuracy and estimated energy. This does not rule out SNN advantages on neuromorphic hardware with on-chip encoding, on temporal inputs such as packet sequences, or with much shorter time windows. It does suggest that SNN papers on flow features should include a spike-input control before crediting membrane dynamics.

Trees remain strong. Untuned XGBoost was the best model at every budget, in line with broader evidence on tabular data [8]. A neuromorphic detector would need to be judged against it, not only against other neural networks.

Limitations. This protocol (v3) was designed after we had seen test results of an earlier version, so these are development results and not an independent confirmation. A fresh held-out dataset is still needed. Feature groups prevent verbatim duplication but not near-duplicates or temporal leakage within an attack episode, and the timestamps of the dataset are unverified. We used three seeds, fixed tree hyperparameters and a single SNN architecture with $T = 12$. Shorter windows, which would cut operation counts in proportion, are the most direct next experiment. Finally, all energy numbers are estimates from operation counts; we made no hardware measurements.

VIII. Conclusion

On CSE-CIC-IDS2018 under a leakage-controlled, budget-matched protocol, an ensemble spiking network detects 96% of attack flows at a 5% false-alarm rate. It beats a parameter-matched perceptron. It loses to gradient-boosted trees, and also to a plain network that is fed the same spike encodings. Much of the remaining error comes from flows that the feature extractor maps to identical vectors under different labels. We hope the protocol, the audit and the released detector make future SNN results on this benchmark easier to compare and harder to overstate.

Reproducibility

Code, configuration, per-run calibration and evaluation records, frozen checkpoints and the Docker detector are available at https://github.com/ozgurucar/ids2018-ensemble-snn. Training ran on a single Kaggle GPU; package versions are pinned in the repository.

References

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[2] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[3] M. Davies *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[4] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.

[5] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *IEEE Conf. Communications and Network Security (CNS)*, 2022, pp. 254–262.

[6] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[7] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium*, 2022, pp. 3971–3988.

[8] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2022.

[9] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.

[10] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[11] J. K. Eshraghian, M. Ward, E. O. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu, "Training spiking neural networks using lessons from deep learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.

[12] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2014, pp. 10–14.

[13] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," *Neural Computation*, vol. 14, no. 1, pp. 21–41, 2002.

[14] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321–1330.

[15] X. Tong, Y. Feng, and J. J. Li, "Neyman-pearson classification algorithms and NP receiver operating characteristics," *Science Advances*, vol. 4, no. 2, p. eaao1659, 2018.

[16] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[17] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[18] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, 2018.